

Task 28 – Production LLM Serving, Asynchronous Microservices & Full-Stack Model Deployment

OBJECTIVE

Construct an end-to-end, production-grade Machine Learning microservice architecture by orchestrating pretrained Large Language Model (LLM) capabilities, engineering asynchronous REST APIs, containerizing execution environments, deploying to cloud infrastructure, and building a full-stack client interface. Interns will implement multi-task model inference pipelines using Hugging Face or direct inference APIs (for sentiment analysis, abstractive summarization, and causal text generation), design a scalable REST API microservice with FastAPI or Flask, containerize the application stack using multi-stage Docker builds, deploy the microservice to cloud hosting platforms (Render, Railway, or Hugging Face Spaces), and integrate an interactive web interface (Streamlit or HTML/CSS/JavaScript) for real-time model interaction, dynamic stress testing, and performance benchmarking.

Total Marks: 100

Part A – Pretrained LLM Inference & Multi-Task Orchestration (20 Marks)

- 1. Multi-Task Inference Pipeline Construction:** Implement modular inference pipelines using local Hugging Face transformers models or official inference APIs capable of serving three distinct production tasks: multi-class Sentiment Analysis, Abstractive Text Summarization, and Causal Text Generation.
- 2. Generation Decoding & Sampling Mechanics:** Formally detail and configure decoding strategies across task endpoints. Compare greedy search, beam search, top-*k* sampling, top-*p* (nucleus) sampling, and temperature scaling, analyzing how each parameter impacts generation diversity, coherence, and inference latency.
- 3. Payload Sanitization & Context Window Management:** Design input processing mechanisms to handle variable token lengths, text truncation, padding strategies, dynamic sequence slicing, and error handling for out-of-bounds context windows across different model architectures.
- 4. Architectural Trade-Off Analysis:** Perform an architectural evaluation comparing direct model weight loading (e.g., PyTorch CPU/GPU instantiation) against remote inference API calls. Contrast memory footprint, computational overhead, latency overhead, cold-start characteristics, and rate-limiting constraints.

Part B – Asynchronous REST API Engineering with FastAPI/Flask (25 Marks)

- 1. Microservice API Design:** Build an asynchronous, production-ready REST API using FastAPI (or Flask with async handlers) exposing three dedicated functional endpoints: `/api/v1/sentiment`, `/api/v1/summarize`, and `/api/v1/generate`.
- 2. Schema Validation & Error Handling:** Implement Pydantic request and response models to enforce strict static typing, input sanitization, and parameter bounds (e.g., `max_tokens`, `temperature`, `top_p`). Configure custom HTTP exception handlers with standardized JSON error responses and explicit status codes (400, 422, 500).
- 3. State Management & Concurrency Optimization:** Implement model singleton lifetime patterns or lazy-loading handlers to prevent redundant model instantiations across API worker processes. Configure asynchronous request execution loops, CORS middleware, structured JSON logging, and a dedicated `/healthz` system monitoring endpoint.

Part C – Containerization & Cloud Infrastructure Deployment (35 Marks)

- 1. Multi-Stage Dockerfile Engineering:** Construct an optimized multi-stage Dockerfile to containerize the ML API microservice. Utilize minimal base images (e.g., python:3.11-slim), isolate dependency resolution layers to optimize build caching for heavy ML libraries (torch, transformers), minimize final layer artifacts, and configure non-root security privileges.
- 2. Local Container Orchestration:** Define local container runtime behavior using docker-compose.yml. Configure environment variable injection, automated health checks, host-to-container port mappings, volume mounts, and explicit CPU/RAM resource quotas.
- 3. Cloud Service Deployment:** Deploy the containerized API microservice to a free public cloud provider (Render, Railway, or Hugging Face Spaces). Document the automated CI/CD deployment pipeline via GitHub integration, configure secure environment secrets, verify live remote endpoint availability, and evaluate cloud execution logs.

Part D – Interactive Web Frontend, Load Testing & System Documentation (20 Marks)

- 1. Interactive Client Interface:** Construct a responsive frontend application using Streamlit or custom HTML/CSS/JavaScript. The interface must communicate asynchronously with the live cloud API endpoints, featuring interactive text inputs, real-time generation parameter controls, dynamic status indicators, and user-facing error reporting.
- 2. Empirical Benchmarking & Load Testing:** Perform load and latency testing on both local and cloud-deployed API instances. Measure and tabulate response latencies (P_{50} , P_{90} , and P_{99} percentiles), RAM/CPU consumption, and request throughput under single-client and concurrent multi-client traffic patterns.
- 3. Engineering Reflections & Bottleneck Resolution:** Document at least two non-trivial architectural or deployment challenges encountered (e.g., container memory allocation limits, dependency bloat, API cold-start timeouts, or concurrency bottlenecks), detailing the technical solutions implemented (such as model quantization, ONNX export, or worker pool adjustment).

Assessment Breakdown

Assessment Area	Marks
Pretrained LLM Inference & Multi-Task Orchestration	20
Asynchronous REST API Engineering with FastAPI/Flask	25
Containerization & Cloud Infrastructure Deployment	35
Interactive Web Frontend, Load Testing & System Documentation	20
Total Marks	100